

AI-Based Adaptive Traffic Signal Control for Intelligent Traffic Management

Ankita Telkar

Department of Computer Science and Engineering
Ballari Institute of Technology and Management
Ballari, Karnataka, India

Abstract—Traffic congestion is a major problem in modern cities owing to rising vehicle density, inefficient signal timing, and fluctuating demand. Conventional fixed-time signals use predetermined phase durations and cannot respond to variations in traffic flow. This paper presents an AI-inspired adaptive traffic signal controller that selects the next green phase and its duration from real-time queue length and traffic density, and formulates the problem as a Markov Decision Process (MDP) that can be extended with reinforcement learning. A Python discrete-time simulation of a four-way intersection compares fixed-time control with the proposed adaptive controller over 30 independent runs in three demand scenarios. Under balanced demand the two methods perform similarly (average waiting time 45.4 s vs. 42.3 s), while under heavily unbalanced demand the adaptive controller reduces average waiting time from 179.2 s to 47.4 s (73.5%), reduces the maximum queue from 674.9 to 181.8 vehicles, and serves about 45% more vehicles. The paper also provides system, algorithm, and MDP flowcharts and outlines extensions using computer vision, deep reinforcement learning, and multi-intersection coordination.

Index Terms—Adaptive Traffic Control, Reinforcement Learning, Markov Decision Process, Intelligent Transportation Systems, Smart Cities, Traffic Simulation

I. INTRODUCTION

Traffic congestion is a major challenge for rapidly growing cities. More vehicles on the road lead to longer delays, longer queues, higher fuel consumption, and increased air pollution. Signalized intersections are critical bottlenecks because conflicting traffic streams must share limited road space in time.

Most deployed signals operate on fixed-time schedules: each approach receives a predefined green interval regardless of demand. This is simple and predictable but inefficient when demand varies by direction or by time of day. Adaptive Traffic Signal Control (ATSC) instead adjusts phase selection and green duration using current traffic measurements, and has become an important topic within Intelligent Transportation Systems (ITS) [1]. Machine learning, and reinforcement learning (RL) in particular, has been widely studied for this purpose [2], [3], [5].

Contributions. The main contributions of this paper are:

- A simple adaptive controller that chooses the next phase and a queue-proportional green time within minimum and maximum bounds.
- A formal MDP formulation (state, action, reward) that links the controller to reinforcement learning.

- Flowcharts of the system architecture, decision algorithm, and RL loop.
- A reproducible Python simulation evaluated over 30 runs in three demand scenarios, reporting mean and standard deviation.

The remainder of the paper is organized as follows. Section II states the problem, Section III reviews related work, Section IV describes the proposed system, Section V formulates the RL model, Section VI presents the simulation and results, and Sections VII–IX discuss limitations, future work, and conclusions.

II. PROBLEM STATEMENT

A fixed-time signal allocates predefined green, yellow, and red durations. Traffic demand, however, is not constant. If the North–South (NS) approach is heavily loaded while the East–West (EW) approach is nearly empty, a fixed-time signal still gives both directions equal green time. This results in:

- increased vehicle waiting time and longer queues,
- wasted green time on empty approaches,
- higher fuel consumption and emissions, and
- congestion that may spill back to upstream intersections.

The objective is therefore to design a controller that allocates green time according to observed demand while respecting minimum and maximum green limits, so as to minimize delay and queue length and maximize throughput.

III. RELATED WORK

A. Signal Control Strategies

Traffic signal control is commonly grouped into three classes. *Fixed-time* control uses predefined phase durations (e.g., 30 s green and 5 s yellow per direction) and cannot react to demand. *Actuated* (sensor-based) control extends or terminates green according to detector calls. *Adaptive* control continuously estimates traffic conditions and optimizes timing online; AI methods are increasingly used to improve these decisions [1].

B. Reinforcement Learning for Signal Control

In RL, an agent observes the state of an environment, selects an action, and receives a reward. Reviews of RL-based signal control describe state representations based on queue length, density, and waiting time, and reward functions based on delay, queue, or pressure [2]. Deep RL methods

including DQN and its variants have been applied to signal timing, including carbon-emission-aware objectives [5], while reliability and safety of learned policies are addressed through formal constraints [3], [4]. Sequence models such as decision transformers have also been explored [6]. Camera-based vehicle detection can supply density estimates to such controllers. Despite promising results, real-world deployment still requires reliable sensing, safety constraints, and extensive validation.

IV. PROPOSED SYSTEM

A. System Architecture

The proposed system has six components: traffic data collection, density calculation, queue estimation, an AI decision module, a signal controller, and performance monitoring. Fig. 1 shows how they form a closed loop.

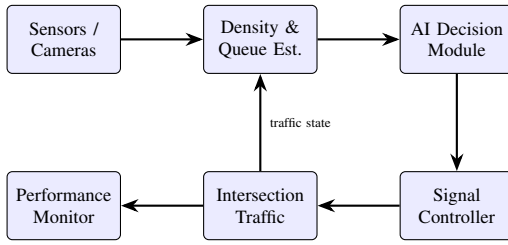


Fig. 1. Closed-loop architecture of the adaptive traffic signal system.

B. Traffic State

At decision time t , the traffic state is

$$S_t = [D_{NS}, D_{EW}, Q_{NS}, Q_{EW}], \quad (1)$$

where D denotes traffic density and Q denotes queue length for the North–South (NS) and East–West (EW) approaches.

C. Decision Rule and Green-Time Allocation

The controller gives the next green phase to the approach with the larger demand. Using queue length as the demand measure (as in the simulation):

$$\text{phase}_{t+1} = \begin{cases} NS, & Q_{NS} \geq Q_{EW}, \\ EW, & \text{otherwise.} \end{cases} \quad (2)$$

The green duration is proportional to the selected queue but bounded:

$$G = \min(G_{\max}, G_{\min} + Q_{\text{sel}}), \quad (3)$$

where Q_{sel} is the queue of the selected approach. The bounds $G_{\min}$ and $G_{\max}$ prevent phase flicker and starvation of the low-demand approach.

D. Algorithm

Fig. 2 presents the decision procedure as a flowchart. The process repeats every simulated second until the horizon is reached.

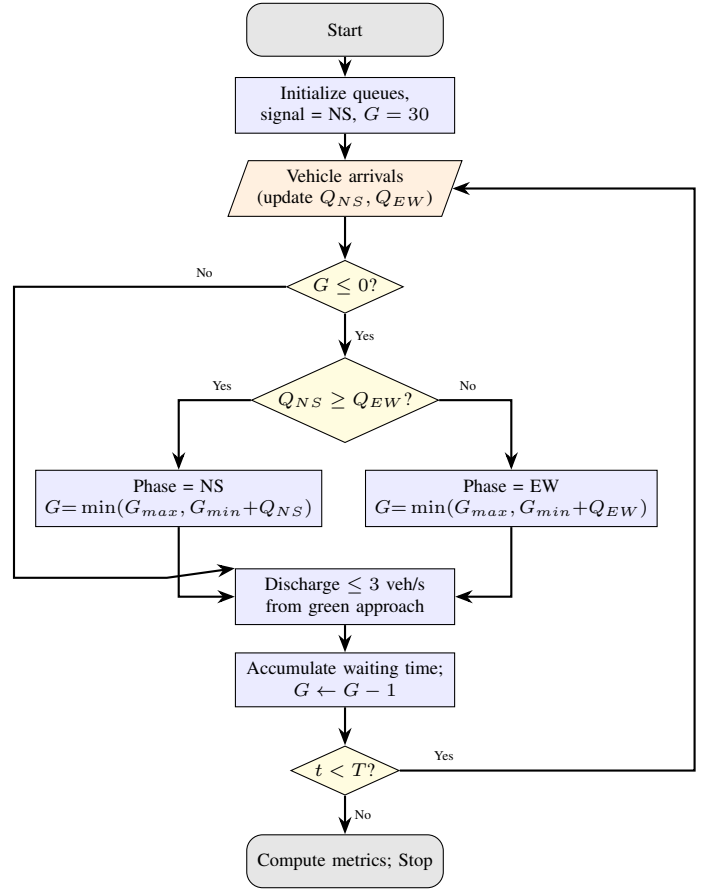


Fig. 2. Flowchart of the adaptive signal control algorithm.

V. REINFORCEMENT LEARNING FORMULATION

Signal control can be modeled as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- **State** $S_t \in \mathcal{S}$: densities and queues as in (1).
- **Action** $A_t \in \mathcal{A}$: choice of signal phase (and optionally its duration).
- **Reward** R_t : measure of traffic performance.
- **Environment**: the signalized intersection and its arrivals.

A simple reward penalizing delay and congestion is

$$R_t = -(\alpha W_t + \beta Q_t), \quad (4)$$

where W_t is total waiting time, Q_t is total queue length, and $\alpha, \beta > 0$ are weights. An emission-aware variant adds a third term:

$$R_t = -(\alpha W_t + \beta Q_t + \gamma E_t), \quad (5)$$

where E_t is an environmental measure such as estimated fuel use or emissions. The learning objective is to find a policy π maximizing the expected discounted return

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \right], \quad 0 < \gamma < 1. \quad (6)$$

The controller evaluated in this paper is a fixed, hand-designed policy that greedily reduces the largest queue; it is not a trained RL agent. Fig. 3 shows the agent–environment loop that a learning-based controller would use in its place.

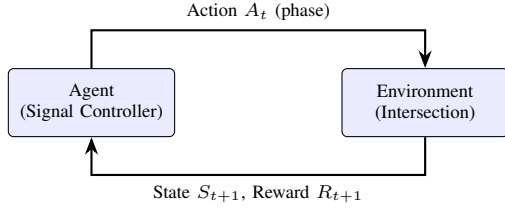


Fig. 3. Agent-environment interaction for RL-based signal control.

VI. SIMULATION AND RESULTS

A. Simulation Setup

A discrete-time (1 s step) simulation models a four-way intersection with two conflicting streams, NS and EW. In each second, each approach receives a random number of arrivals uniformly distributed in $\{0, \dots, a\}$. The green approach discharges at most 3 vehicles per second. A 3 s yellow/all-red lost time is applied whenever the phase changes, for both controllers, so that switching is not free. Table I lists the parameters. Both controllers use identical random seeds in each run, so they face the same arrival sequences.

TABLE I
SIMULATION PARAMETERS

Parameter	Value
Simulation time	600 s
Fixed-time green (each phase)	30 s
Adaptive min / max green	10 s / 40 s
Lost time per phase change	3 s
Discharge rate	3 veh/s
Arrivals per second	Uniform $\{0, \dots, a\}$
Independent runs per scenario	30

Three demand scenarios are used (arrival bound a for NS/EW): **S1** balanced (3/3), **S2** moderate imbalance (4/2), and **S3** heavy imbalance (5/1).

B. Implementation

The core simulation is listed below (the full script also loops over 30 seeds and scenarios and reports mean and standard deviation).

```

1 import random
2
3 SIM_TIME, MIN_GREEN, MAX_GREEN = 600, 10, 40
4 FIXED_GREEN, YELLOW, DISCHARGE = 30, 3, 3
5
6 def simulate(adaptive, arr_ns, arr_ew, rng):
7     q = {"NS": 0, "EW": 0}
8     arr = {"NS": arr_ns, "EW": arr_ew}
9     wait = served = max_q = 0
10    signal, green, yellow = "NS", FIXED_GREEN, 0
11
12    for t in range(SIM_TIME):
13        for d in q:
14            q[d] += rng.randint(0, arr[d])
15
16        if yellow > 0:
17            yellow -= 1
18        elif green <= 0:
19            # choose phase
20            if adaptive:
21                new = "NS" if q["NS"] >= q["EW"] else "EW"
22                green = min(MAX_GREEN, MIN_GREEN + q[new])
23            else:
24                new = "EW" if signal == "NS" else "NS"
25                green = FIXED_GREEN
26                if new != signal:
27                    yellow = YELLOW
28                    signal = new
29
30        if yellow == 0 and green > 0:
31            # discharge
32            p = min(q[signal], DISCHARGE)
33            q[signal] -= p
34            served += p
35            green -= 1
36
37        wait += q["NS"] + q["EW"]
38        max_q = max(max_q, max(q.values()))
39
40    return wait / max(served, 1), max_q, served

```

```

23 new = "EW" if signal == "NS" else "NS"
24 green = FIXED_GREEN
25 if new != signal:
26     yellow = YELLOW
27     signal = new
28
29 if yellow == 0 and green > 0:
30     # discharge
31     p = min(q[signal], DISCHARGE)
32     q[signal] -= p
33     served += p
34     green -= 1
35
36 wait += q["NS"] + q["EW"]
37 max_q = max(max_q, max(q.values()))
38
39 return wait / max(served, 1), max_q, served

```

C. Performance Metrics

Average waiting time (AWT) is total vehicle-seconds spent in queue divided by vehicles served:

$$AWT = \frac{T_W}{N}, \quad (7)$$

with $T_W = \sum_t (Q_{NS}(t) + Q_{EW}(t))$ and N the number of vehicles served. **Maximum queue length** is the largest single-approach queue observed during the run. **Vehicles served** is the total throughput in 600 s. Because queues are not cleared at the end of the run, AWT is a throughput-normalized delay index rather than a per-vehicle travel-time measurement.

D. Results

Table II reports mean $\pm$ standard deviation over 30 runs, and Fig. 4 compares average waiting time across scenarios.

TABLE II
PERFORMANCE COMPARISON (MEAN $\pm$ STD, 30 RUNS)

Scenario	Method	AWT (s)	Max queue	Served
S1	Fixed	45.4 $\pm$ 8.2	137.7 $\pm$ 23.3	1589.3 $\pm$ 9.4
	Adaptive	42.3 $\pm$ 7.0	131.3 $\pm$ 19.3	1612.9 $\pm$ 14.5
S2	Fixed	88.9 $\pm$ 8.6	397.5 $\pm$ 38.2	1384.1 $\pm$ 24.5
	Adaptive	44.0 $\pm$ 6.6	147.7 $\pm$ 21.3	1617.0 $\pm$ 19.6
S3	Fixed	179.2 $\pm$ 11.3	674.9 $\pm$ 36.0	1107.0 $\pm$ 15.2
	Adaptive	47.4 $\pm$ 4.9	181.8 $\pm$ 17.0	1602.9 $\pm$ 26.3

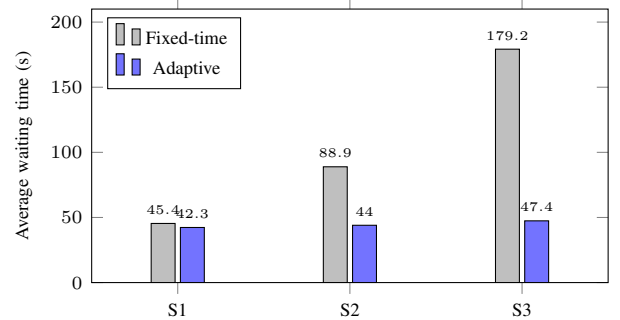


Fig. 4. Average waiting time by demand scenario (mean of 30 runs).

E. Discussion

Under balanced demand (S1) the benefit is small: AWT improves by about 7% (45.4 to 42.3 s), which is within roughly one standard deviation, so the difference should not be considered conclusive. As imbalance grows, fixed-time control degrades sharply because half of the green time serves the lighter approach while the heavier approach's queue grows. The adaptive controller's AWT stays near 42–47 s in all scenarios. In S2 and S3 it reduces AWT by 50.5% and 73.5%, reduces the maximum queue by 62.8% and 73.1%, and serves 16.8% and 44.8% more vehicles, respectively. These results are consistent with the expectation that demand-responsive green allocation matters most when demand is uneven [1].

The comparison is against a simple fixed-time baseline with an equal split, not against a tuned or actuated controller, so the gains represent an upper-bound view of adaptivity. A tuned fixed-time plan with a longer green for the heavy approach would narrow the gap.

VII. ADVANTAGES AND LIMITATIONS

A. Advantages

The controller responds to current demand, reduces wasted green time, limits queue growth, and can incorporate real-time sensor or camera data. It is computationally trivial, interpretable, and extensible to learning-based and multi-intersection control.

B. Limitations

The simulation simplifies vehicle motion (constant discharge rate, no acceleration or platooning), has no road geometry, turning movements, pedestrians, emergency vehicles, or weather, and uses synthetic uniform arrivals rather than measured data. It has not been tested on a physical signal. The controller is an AI-inspired heuristic rather than a trained deep RL model, and only two phases are modeled. Real-world deployment would require extensive safety testing and validation [4].

VIII. FUTURE SCOPE

- **Computer vision:** camera-based vehicle detection and density estimation.
- **Deep RL:** DQN, PPO, or transformer-based policies [5], [6] trained against the reward in (6)–(7).
- **Multiple intersections:** coordinated control along corridors and networks.
- **Emergency vehicle priority:** detection and controlled preemption.
- **Environmental optimization:** emission- and fuel-aware rewards [5].
- **Realistic simulation:** coupling with SUMO and evaluation on measured traffic data.

IX. CONCLUSION

This paper presented an adaptive traffic signal controller with an MDP formulation, system and algorithm flowcharts, and a reproducible Python evaluation. Across 30 runs per scenario, the adaptive controller matched fixed-time performance under balanced demand and substantially outperformed it under unbalanced demand, cutting average waiting time by up to 73.5% and maximum queue length by up to 73.1% while increasing throughput. Because the evaluation uses a simplified synthetic model and a basic baseline, the results are indicative rather than definitive; future work will replace the heuristic with learned policies and validate them in a microscopic simulator such as SUMO and on real traffic data.

REFERENCES

- [1] A. Agrahari, M. M. Dhabu, P. S. Deshpande, A. Tiwari, M. A. Baig, and A. D. Sawarkar, "Artificial Intelligence-Based Adaptive Traffic Signal Control System: A Comprehensive Review," *Electronics*, vol. 13, no. 19, 2024.
- [2] P. Michailidis, I. Michailidis, C. R. F. Lazaridis, and E. Kosmatopoulos, "Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations," *Infrastructures*, vol. 10, no. 5, 2025.
- [3] "VIA AI: Reliable Deep Reinforcement Learning for Traffic Signal Control," in *Proc. IEEE Int. Conf. Data Mining Workshops*, 2024.
- [4] "Formally Constrained Reinforcement Learning for Traffic Signal Control at Intersections," in *Proc. IEEE Int. Systems Conf.*, 2025.
- [5] J. Lv, Z. Wang, and J. Ma, "Carbon Emission Reduction in Traffic Control: A Signal Timing Optimization Method Based on Rainbow DQN," *Applied Sciences*, vol. 15, no. 3, 2025.
- [6] "Sequence Decision Transformer for Adaptive Traffic Signal Control," *Sensors*, vol. 24, no. 19, 2024.